RMBench: Memory-Dependent Robotic Manipulation Benchmark with Insights into Policy Design

Tianxing Chen [*1] **Yuran Wang** [*2 3] **Mingleyang Li** [*2] **Yan Qin** [*4] **Hao Shi** [5] **Zixuan Li** [6]
Yifan Hu [2] **Yingsheng Zhang** [5] **Kaixuan Wang** [1] **Yue Chen** [2] **Hongcheng Wang** [2] **Renjing Xu** [4]
Ruihai Wu [2] **Yao Mu** [7] **Yaodong Yang** [2 3] **Hao Dong** [† 2] **Ping Luo** [† 1]
∗ Equal contribution, † Corresponding authors
https://RMBench.github.io

Abstract

Robotic manipulation policies have made rapid progress in recent years, yet most existing approaches give limited consideration to memory capabilities. Consequently, they struggle to solve tasks that require reasoning over historical observations and maintaining task-relevant information over time, which are common requirements in real-world manipulation scenarios. Although several memory-aware policies have been proposed, systematic evaluation of memory-dependent manipulation remains underexplored, and the relationship between architectural design choices and memory performance is still not well understood. To address this gap, we introduce **RMBench**, a simulation benchmark comprising 9 manipulation tasks that span multiple levels of memory complexity, enabling systematic evaluation of policy memory capabilities. We further propose **Mem-0**, a modular manipulation policy with explicit memory components designed to support controlled ablation studies. Through extensive simulation and real-world experiments, we identify memory-related limitations in existing policies and provide empirical insights into how architectural design choices influence memory performance.

1. Introduction

Recent progress in robotic manipulation has demonstrated strong capabilities across a wide range of tasks. Modern policies such as Pi0.6 (Intelligence et al., 2025a) and RDT2 (Team, 2025) achieve impressive performance in flexible object manipulation and fine-grained skills, including complex activities like coffee making. Nevertheless, most existing robotic policies are primarily designed for short-horizon tasks and fine-grained manipulation. These approaches typically rely on a fixed-length window of recent observations, implicitly assuming that the underlying decision process is approximately Markovian.

In contrast, memory-dependent tasks are ubiquitous in real-world robotic applications. Such tasks are inherently non-Markovian, as past observations and actions may influence future decisions over extended temporal horizons. Examples include remembering the location of previously placed objects or reasoning over multiple attempts after forgetting a password. These tasks are both challenging and practically important, as they require robots to retain, retrieve, and utilize information beyond short-term sensory inputs.

Motivated by this challenge, several recent works have begun to explore memory-aware robotic policies. Approaches such as MemoryVLA (Shi et al., 2025), MemER (Sridhar et al., 2025), CronusVLA (Li et al., 2025), and SAM2Act (Fang et al., 2025) incorporate explicit memory mechanisms to address long-horizon and memory-dependent decision making. Despite these efforts, the field currently lacks a systematically designed experimental platform for evaluating and analyzing robotic policies under long-term memory requirements. In particular, there is limited understanding of the underlying mechanisms that make memory strategies effective for robotic manipulation.

Existing benchmarks only partially bridge this gap. MemoryBench (Fang et al., 2025) comprises seven single-arm manipulation tasks that involve memory, yet only three can be reliably reproduced in simulation, and the benchmark provides limited guidance on principled task design. MIKASA (Cherepanov et al., 2025) introduces 32 memory-related manipulation tasks, but its formulation is largely tailored to reinforcement learning rather than general imitation learning. LIBERO-Long (Liu et al., 2023) offers ten long-horizon tasks, though these tasks do not explicitly

[1]MMLab@HKU [2]PKU [3]PsiBot [4]HKUST (GZ) [5]THU [6]SZU [7]SJTU. Correspondence to: Tianxing Chen <chentianxing@connect.hku.hk>, Ping Luo <pluo@hku.hk>, Hao Dong <hao.dong@pku.edu.cn>.

Preprint. February 21, 2026.

demand memory since all task-relevant information remains observable throughout execution.

To address these limitations, we first introduce Task Memory Complexity, a principled metric for characterizing memory requirements in robotic manipulation tasks. This metric provides a systematic way to classify memory-dependent tasks and serves as a guideline for task design. Based on this formulation, we propose **RMBench**, a robotic manipulation benchmark built on RoboTwin 2.0 platform. RMBench consists of 9 dual-arm manipulation tasks spanning different levels of task memory complexity, enabling large-scale and controlled studies of memory retention and utilization in robotic manipulation.

Furthermore, we propose **Mem-0**, a novel memory-oriented robotic policy designed with modular components that can be easily replaced or reconfigured. Mem-0 adopts a dual-system architecture with a task-phase classifier that explicitly distinguishes different stages of a task, allowing structured memory usage across long horizons. Through systematic ablation studies of Mem-0, we analyze which design components are critical for effective memory in robotic manipulation and derive insights for future policy design.

Our main contributions are summarized as follows:

- We introduce Task Memory Complexity, a principled metric for categorizing memory-dependent robotic manipulation tasks, and propose **RMBench**, a simulation benchmark comprising 9 memory-centric tasks based on this metric.
- We propose **Mem-0**, a memory-oriented robotic manipulation policy featuring a dual-system architecture with a task-phase classifier for flexible memory usage.
- We conduct comprehensive evaluations of representative state-of-the-art policies on RMBench and perform detailed ablation studies of Mem-0, revealing which policy design mechanisms are most beneficial for memory in robotic manipulation.

2. Related Work

2.1. Robotic Manipulation Benchmarks

Physics-based simulators underpin modern robotic manipulation research, and numerous simulation benchmarks have been proposed in recent years. RoboTwin (Mu et al., 2025; Chen et al., 2025a), RoboCasa (Nasiriany et al., 2024), ManiSkill3 (Tao et al., 2025), AutoBio (Lan et al., 2025), UniVTAC (Chen et al., 2026), DexGarmentLab (Wang et al., 2025), BEHAVIOR-1K (Li et al., 2024a), and SIMPLER (Li et al., 2024c) provide diverse manipulation tasks, yet most scenarios emphasize short-horizon interactions or can be solved without relying on historical observations. Several benchmarks have begun to consider memory-related manipulation tasks. MemoryBench (Fang et al., 2025) includes a limited number of memory-dependent tasks but suffers from poor reproducibility and lacks clear task design principles. MIKASA (Cherepanov et al., 2025) introduces a larger collection of memory-related tasks, though its design is primarily tailored to reinforcement learning. LIBERO-Long (Liu et al., 2023) and RoboCerebra (Han et al., 2025) features long-horizon tasks, but task-relevant information remains observable throughout execution and therefore does not explicitly require memory. In contrast, RMBench explicitly stratifies memory requirements in manipulation tasks, enabling systematic analysis of memory-based policies across different levels of task difficulty.

2.2. Robotic Manipulation Policies

Recent advances in generative models and imitation learning have produced a wide range of robotic manipulation policies that achieve strong performance on individual tasks (Zhao et al., 2023; Chi et al., 2025; Ze et al., 2024; Chen et al., 2025b; Lu et al., 2024; Su et al., 2025). Inspired by large visual foundation models, many approaches adopt Vision-Language-Action (VLA) formulations (Wen et al., 2025a; Lin et al., 2025; Liang et al., 2025; Shen et al., 2025; Wen et al.; 2025b), where policies are pretrained on large-scale robot datasets and exhibit improved generalization. Representative examples include Pi0.5 and Pi0.6 (Intelligence et al., 2025b;a), RDT2 (Team, 2025), and X-VLA (Zheng et al., 2025), as well as methods that incorporate future observation prediction such as Motus (Bi et al., 2025), CogACT (Li et al., 2024b), and CronusVLA (Li et al., 2025).

Despite these advances, most existing policies rely on fixed-length observation histories, which limits their ability to selectively retain task-relevant information over long time horizons. This limitation motivates recent memory-aware approaches, including MemoryVLA (Shi et al., 2025), MemER (Sridhar et al., 2025), and SAM2Act (Fang et al., 2025), which explicitly incorporate memory mechanisms for memory-dependent manipulation. Building on this line of work, we propose Mem-0, a modular memory-enabled policy designed to facilitate systematic ablation and analysis of memory components and architectural choices.

3. RMBench

In this section, we introduce **Task Memory Complexity**, a principled criterion for characterizing memory requirements in robotic manipulation tasks and guiding benchmark design. Based on this, we propose **RMBench**, a simulation benchmark comprising nine manipulation tasks with varying memory demands, designed to support controlled evaluation across different levels of task memory complexity.

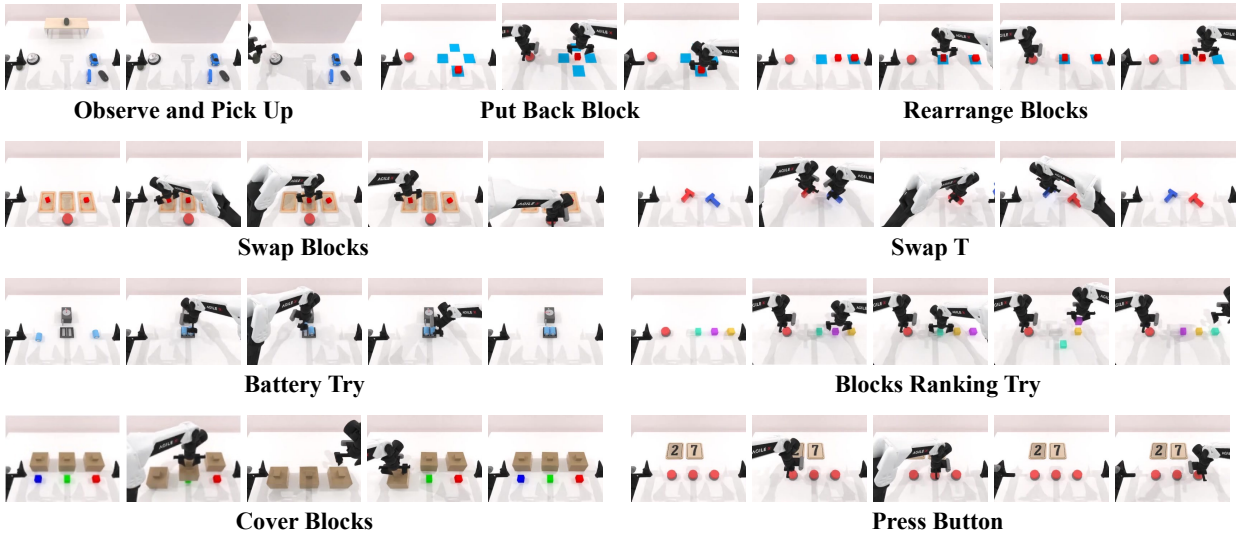

Figure 1. **RMBench Tasks.** We illustrate the nine memory-dependent tasks in RMBench along with their key execution steps. Tasks detailed description are shown in Appendix. A.

3.1. Task Memory Complexity (TMC)

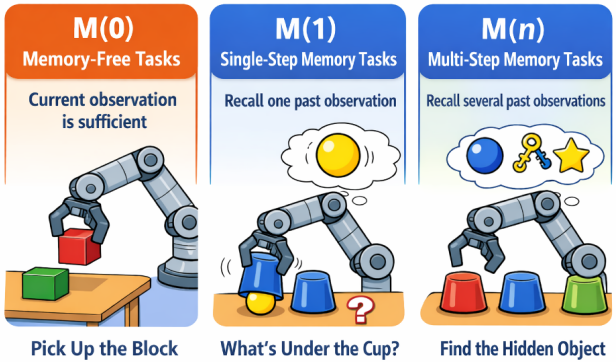

Figure 2. **Task Memory Complexity.**

Robotic manipulation often operates under partial observability, where the current observation alone may be insufficient to determine task progress or the correct next action without access to past information. Such partial observability may arise from occlusions, delayed effects, or state aliasing. Importantly, the required past information does not necessarily correspond to a contiguous sequence of recent observations, but rather to a small set of task-relevant observations occurring at arbitrary time steps. Existing benchmarks typically assess memory through specific policy architectures, but lack a task-centric criterion that characterizes how much task-relevant information must be retained over time. To address this gap, we introduce **Task Memory Complexity (TMC)**, which measures the minimal amount of past information required for optimal decision-making.

Setup. We model a manipulation task as a partially observable Markov decision process (POMDP) with latent state $s_t \in \mathcal{S}$, observation $o_t \in \mathcal{O}$, and action $a_t \in \mathcal{A}$. Let the full interaction history up to time t be

$$h_t = (o_{1:t}, a_{1:t-1}). \tag{1}$$

Rather than assuming access to the full history, we consider a memory state that summarizes task-relevant information from past observations. Let $\mathcal{M}_t$ denote a memory representation constructed from h_t, and let $\mathcal{M}_t^{(k)}$ denote a memory state that encodes information from at most k task-relevant past observations.

Definition. **Task Memory Complexity** is defined as the smallest integer $m \geq 0$ such that there exists an optimal policy π^* whose action at time t depends only on the memory state $\mathcal{M}_t^{(m)}$. Formally,

$$\exists\, \pi^* \text{ s.t. } \pi^*(a_t \mid h_t) = \pi^*(a_t \mid \mathcal{M}_t^{(m)}), \quad \forall t. \tag{2}$$

Task Annotation. Tasks are annotated according to their Task Memory Complexity using the notation $M(0)$, $M(1)$, or more generally $M(n)$, where the index indicates the number of task-relevant past observations that must be retained to solve the task optimally.

Interpretation. $M(0)$ denotes memory-free tasks, for which the current observation is sufficient to uniquely determine task progress and the optimal action. $M(1)$ denotes tasks that require retaining a single task-relevant past observation to disambiguate the current state. More generally, $M(n)$ denotes tasks whose optimal decisions depend on retaining n task-relevant past observations, capturing non-local and multi-step temporal dependencies.

3.2. RMBench System Design

RMBench is developed within the RoboTwin 2.0 (Chen et al., 2025a) system framework. It is built on top of the SAPIEN (Xiang et al., 2020) simulation engine and supports both automated data synthesis and integrated policy evaluation within a unified pipeline. This design enables scalable data generation as well as consistent and reproducible benchmarking of robotic manipulation policies.

In addition, we provide fine-grained language annotations that align with each action–observation pair. These annotations assign explicit linguistic descriptions to low-level interactions and state transitions, offering structured and dense supervision signals for downstream training of high-level reasoning or memory modules.

3.3. RMBench Benchmark Tasks

Based on the proposed Task Memory Complexity (Sec. 3.1), we design a total of nine memory-dependent manipulation tasks. These tasks are grouped into two categories: five $M(1)$ tasks, and four $M(n)$ tasks. Representative key frames for each task are illustrated in Fig. 2, and detailed task specifications are provided in the Appendix A.

The $M(1)$ tasks consist of *Observe and Pick Up*, *Rearrange Blocks*, *Put Back Block*, *Swap Blocks* and *Swap T*. These tasks require the policy to retain a single past observation or a fixed, limited number of historical frames. Successful execution depends on dynamically attending to task-relevant information across different stages of the task.

The $M(n)$ tasks include *Blocks Ranking Try*, *Press Button*, *Cover Blocks*, and *Battery Try*. These tasks require repeated active exploration, trial-and-error interactions, or repeated execution for a task-specific number of attempts, often guided by external feedback. As a result, they demand strong long-term memory retention and effective retrieval mechanisms to accumulate and utilize historical information over extended horizons.

4. Mem-0 Policy

In this section, we present **Mem-0**, a modular memory-oriented robotic policy designed for systematic analysis of memory mechanisms. As shown in Fig. 3, Mem-0 consists of a **Planning Module** that performs subtask-level reasoning based on key memory and an **Execution Module** that executes subtasks using sliding-window and anchor memories. The two modules are connected by a **Subtask End Classifier**, which enables closed-loop planning and execution. This modular design supports fine-grained analysis of the roles of different memory components in memory-dependent robotic manipulation.

4.1. Planning Module

In long-horizon manipulation tasks, end-to-end VLA models are prone to error accumulation and trajectory drift during inference. Prior work (Sridhar et al., 2025; Wen et al., 2024) mitigates these issues through subtask decomposition. However, such approaches become insufficient in memory-dependent settings, such as the $M(n)$-type tasks in RMBench, where accurate subtask inference requires reasoning over multiple previously completed subtasks, for example by tracking the number of button presses. To address this limitation, we introduce a key memory window module within the Planning Module, which aggregates completed subtasks and enables memory-aware subtask reasoning.

The Planning Module performs subtask-level reasoning using a vision language model conditioned on visual observations and structured memory. At planning step t, the model receives as input the initial observation o_0, the task goal g, and the memory state $\mathcal{M}_{t-1}$, and predicts the next subtask as

$$s_t = \mathcal{V}_{\text{plan}}(o_0,\ g,\ \mathcal{M}_{t-1})\,. \tag{3}$$

Here, $o_0 \in \mathbb{R}^{H\times W\times 3}$ denotes the initial RGB observation at the beginning of an episode, and $g \in \mathcal{T}$ denotes the global task instruction. The finished-task memory $\mathcal{M}_{t-1}$, also referred to as the key memory window, aggregates all previously completed subtasks and is defined as

$$\mathcal{M}_{t-1} = \left\{\left(s_i,\ o_i^{\text{end}}\right)\right\}_{i=1}^{t-1}, \tag{4}$$

where $s_i \in \mathcal{T}$ is the textual description of the i-th subtask and $o_i^{\text{end}} \in \mathbb{R}^{H\times W\times 3}$ is the RGB observation at the termination of that subtask.

Conditioning on $\mathcal{M}_{t-1}$ enables the vision–language model to explicitly reason over previously executed subtasks and their corresponding visual outcomes, thereby supporting memory-dependent subtask inference beyond single-frame observation-based planning.

In contrast to existing approaches that infer a new subtask at every observation frame and thus require $O(T)$ planning calls over a horizon of T timesteps, the Planning Module performs subtask reasoning only upon subtask termination, as identified by a Subtask End Classifier (Section 4.3). For a long-horizon task consisting of N subtasks, where $N \ll T$, this design reduces the number of planning invocations to $O(N)$. As a result, the Execution Module can operate at a high control frequency within each subtask without being constrained by planning latency, substantially reducing computational overhead and accelerating task execution.

4.2. Execution Module

Although subtask-level planning is effective for many memory-dependent tasks, some tasks are not well suited

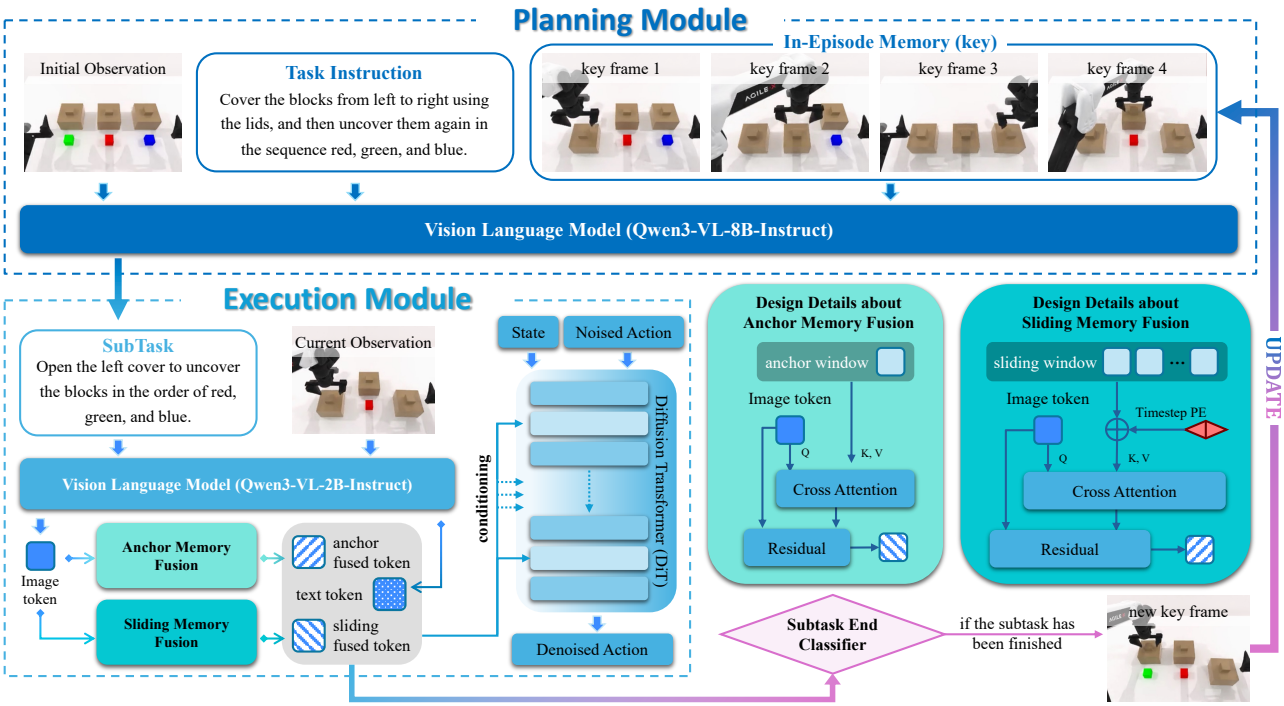

Figure 3. **Mem-0 Pipeline**. Mem-0 comprises a Planning Module and an Execution Module linked by a Subtask End Classifier. The Planning Module generates high-level subtasks from task instructions, observations, and key-frame memory, while the Execution Module produces low-level actions using the current observation, the subtask, and fused anchor and sliding memories in a diffusion-based policy. Upon subtask completion, a key frame is stored to enable iterative planning and execution until task completion.

for explicit subtask decomposition. For instance, in $M(1)$-type tasks such as *Swap T*, the target placement orientation of object T cannot be specified reliably through language. In addition, overly fine-grained subtask decomposition increases annotation effort and planning latency, which degrades overall execution efficiency. To address these limitations, we incorporate an **anchor memory module** and a **sliding memory window module** within the Execution Module. This design allows the Execution Module to handle $M(1)$-type tasks without additional subtask decomposition by maintaining a persistent anchor memory together with short-term transient memories.

The **Execution Module** executes the current subtask using a diffusion-based policy conditioned on multimodal perception and memory. A vision–language model (VLM), denoted by $\mathcal{V}_{\text{exec}}(\cdot)$, encodes the current RGB observation $o_t \in \mathbb{R}^{H\times W\times 3}$ and the subtask instruction $s_t \in \mathcal{T}$ into image and text token embeddings:

$$\left(\mathbf{Z}_t^{\text{img}}, \mathbf{Z}_t^{\text{text}}\right) = \mathcal{V}_{\text{exec}}(o_t, s_t). \tag{5}$$

Mean pooling is then applied to obtain compact latent representations $\mathbf{z}_t^{\text{img}} = \text{MeanPool}(\mathbf{Z}_t^{\text{img}})$ and $\mathbf{z}_t^{\text{text}} = \text{MeanPool}(\mathbf{Z}_t^{\text{text}})$.

To incorporate memory, the image latent attends to two memory buffers: an *anchor memory* $\mathcal{A}$ and a *sliding memory window* $\mathcal{S}_t$. Memory-conditioned representations are computed via cross-attention:

$$\tilde{\mathbf{z}}_t^{l} = \text{CrossAttn}\left(\mathbf{z}_t^{\text{img}}, \mathcal{M}_t^{m}\right) + \mathbf{z}_t^{\text{img}}, \tag{6}$$

where $l \in \{\text{anchor}, \text{slide}\}$, $\mathcal{M}_t^{\text{anchor}} = \mathcal{A}$ and $\mathcal{M}_t^{\text{slide}} = \mathcal{S}_t$. The fused token are concatenated with the text token to form the conditioning vector $\mathbf{c}_t = [\, \tilde{\mathbf{z}}_t^{\text{anchor}}; \tilde{\mathbf{z}}_t^{\text{slide}}; \mathbf{z}_t^{\text{text}} \,]$.

After attention at timestep t, the image latent is appended to the sliding memory window:

$$\mathcal{S}_{t+1} = \text{Trunc}_K\left(\mathcal{S}_t \cup \{\mathbf{z}_t^{\text{img}}\}\right) \tag{7}$$

where $\text{Trunc}_K(\cdot)$ retains the most recent K elements. At the beginning of a subtask ($t = 0$), the image latent $\mathbf{z}_0^{\text{img}}$ is stored as the anchor memory $\mathcal{A} = \{\mathbf{z}_0^{\text{img}}\}$, which remains fixed throughout the subtask. Upon subtask termination, both memory buffers are reset: $\mathcal{A} \leftarrow \varnothing, \mathcal{S}_t \leftarrow \varnothing$.

For action generation, we employ a diffusion transformer (DiT) with a fixed action horizon $H = 30$. Let $\mathbf{a}_{t:t+H-1}^{\epsilon} \in \mathbb{R}^{H\times d_a}$ denote a noisy action sequence obtained by perturbing a ground-truth action sequence with Gaussian noise, and let $\hat{\mathbf{a}}_{t:t+H-1} \in \mathbb{R}^{H\times d_a}$ denote the corresponding denoised prediction produced by the model. At timestep t, the DiT predicts a denoised action sequence

$$\hat{\mathbf{a}}_{t:t+H-1} = \text{DiT}\left(\mathbf{a}_{t:t+H-1}^{\epsilon}, \mathbf{x}_t, \mathbf{c}_t\right). \tag{8}$$

A prefix of the predicted sequence, denoted as $\hat{\mathbf{a}}_{t:t+\Delta-1}$ with $1 \leq \Delta \leq H$, is executed as control commands before the next replanning step.

4.3. Subtask End Classifier

To enable closed-loop interaction between the Planning and Execution Modules, we introduce a *Subtask End Classifier* to detect subtask completion. The classifier is implemented as a lightweight multilayer perceptron (MLP) that operates on the conditioning vector $\mathbf{c}_t$ and outputs a binary signal $\mathcal{C}_{\text{end}}(\mathbf{c}_t) \in \{0, 1\}$, indicating whether the current subtask is ongoing or terminated at timestep t.

To improve robustness and avoid premature termination caused by transient noise, we enforce a temporal consistency criterion. A subtask is considered complete only if the classifier predicts termination for $L = 8$ consecutive timesteps:

$$\sum_{i=t-L+1}^{t} \mathcal{C}_{\text{end}}(\mathbf{c}_i) = L. \tag{9}$$

Once this condition is satisfied, the subtask is terminated, and the final observation o_t^{end} together with the corresponding subtask description s_t is passed to the Planning Module to trigger the next round of subtask-level reasoning. This mechanism establishes a closed loop between high-level planning and low-level execution, enabling coordinated and iterative subtask inference and execution.

5. Experiment

We design a set of experiments to validate three key objectives: (1) to evaluate the performance of existing manipulation policies and Mem-0 on RMBench, thereby characterizing their ability to handle memory-dependent tasks across different levels of difficulty; (2) to conduct systematic ablation studies on the Mem-0 architecture in order to analyze how different module designs affect performance on memory-intensive manipulation tasks; and (3) to perform real-world robotic experiments to assess the effectiveness and generalization of Mem-0 beyond simulation.

5.1. Evaluation of Policies on RMBench

we benchmark a diverse set of policies on RMBench, including non-pretrained methods, pretrained methods, and Mem-0, our memory-centric policy. Specifically, DP and ACT are used as non-pretrained baselines, while Pi0.5 and X-VLA represent pretrained approaches. For each task under both the $M(1)$ and $M(n)$ settings, all models are trained with 50 expert demonstrations and evaluated over 100 rollout episodes.

For $M(1)$-type tasks, Mem-0 operates without subtask decomposition, and the reported results primarily reflect the capability of the Execution Module. In contrast, for $M(n)$-type tasks, subtask decomposition is performed at key decision points, and the results capture the joint performance of the Planning and Execution Modules. All baseline methods are trained without subtask decomposition. We report success rates for all methods in Table 1.

Experimental results show that both non-pretrained and pretrained baselines consistently underperform on memory-dependent tasks. This behavior can be attributed to the fact that most existing models are designed under a Markovian assumption, where the next action is determined solely by the current observation. When applied to the non-Markovian tasks in RMBench, these models fail to infer the correct action without access to task-relevant past information, leading to substantial performance degradation. Representative failure cases of baseline methods are illustrated in Fig. 4.

In contrast, Mem-0 demonstrates substantial performance gains across the majority of tasks, indicating the effectiveness of explicitly incorporating memory mechanisms. On average, Mem-0 improves success rates by 38.4% on $M(1)$ tasks and 21.2% on $M(n)$ tasks relative to the baselines, underscoring the critical role of memory modules in addressing memory-dependent manipulation in RMBench.

Despite these gains, Mem-0 exhibits limitations on tasks that require strong semantic understanding, such as *Observe and Pick Up*, where pretrained models retain an advantage due to large-scale pretraining. In fine-grained manipulation tasks such as *Swap T*, Mem-0 exhibits limited placement accuracy, resulting in only marginal performance gains. In the *Press Button* task, the small magnitude of individual press actions further complicates reliable termination detection: the Subtask End Classifier may fail to consistently recognize task completion, causing repeated presses or missed contacts and ultimately zero success.

These results highlight several open challenges in the current Mem-0 design. More Visualization and Analysis can be found in the Appendix C and Supplementary Material. Nevertheless, the overall performance trends clearly demonstrate that explicit memory modeling yields significant improvements on the majority of memory-dependent tasks in RMBench.

5.2. Analysis on Memory-Related Module

In this section, we analyze the contribution of individual components in Mem-0 to provide insights into effective memory module design. To this end, we conduct four ablation studies:

(1) **w/o Anchor**: The anchor memory module in the Execution Module is removed, so image tokens are no longer fused with anchor memory tokens.

Table 1. **RMBench benchmark results.** RMBench includes nine manipulation tasks across the $M(1)$ and $M(n)$ levels of Task Memory Complexity. We report success rates for five policies, each trained with 50 synthesized demonstrations and evaluated over 100 rollouts. (**Bold** : best; Underlined: second-best; Green: relative improvement over the second-best).

Tasks	TMC	DP	ACT	Pi0.5	X-VLA	Mem-0 (ours)
Observe and Pick Up	$M(1)$	1%	1%	**9%**	**9%**	4%
Rearrange Blocks	$M(1)$	0%	29%	13%	13%	**89%**
Put Back Block	$M(1)$	0%	0%	11%	18%	**90%**
Swap Blocks	$M(1)$	11%	2%	24%	16%	**67%**
Swap T	$M(1)$	**20%**	2%	15%	3%	14%
Average	$M(1)$	6.4%	6.8%	14.4%	11.8%	**52.8%(+38.4%)**
Battery Try	$M(n)$	10%	19%	16%	26%	**28%**
Blocks Ranking Try	$M(n)$	10%	0%	6%	1%	**18%**
Cover Blocks	$M(n)$	0%	0%	0%	2%	**68%**
Press Button	$M(n)$	0%	0%	0%	0%	0%
Average	$M(n)$	5%	4.8%	5.5%	7.3%	**28.5%(+21.2%)**
Total Average	/	5.8%	5.9%	10.4%	9.8%	**42.0%(+31.6%)**

Table 2. **Ablation Studies.** (**Bold** : the best results; Underlined: the second-best results).

$M(1)$ Tasks	Observe and Pick Up	Rearrange Blocks	Put Back Block	Swap Blocks	Swap T	***Average***
Vanilla (ours)	**4%**	**89%**	**90%**	**67%**	14%	**52.8%**
w/o Anchor	**4%**	73%	35%	15%	7%	26.8%
w/o Sliding	3%	62%	78%	39%	**20%**	40.4%

$M(n)$ Tasks	Battery Try	Blocks Ranking Try	Cover Blocks	Press Button	***Average***
Vanilla (ours)	28%	18%	68%	0%	28.5%
w/o Key	13%	1%	5%	0%	4.8%
w/o Anchor	14%	0%	**92%**	1%	26.8%
w/o Sliding	17%	0%	84%	0%	25.3%
GT Classifier	**30%**	**45%**	**92%**	**14%**	**45.3%**

(2) **w/o Sliding**: The sliding memory module in the Execution Module is removed, and image tokens are not fused with historical sliding memory tokens.

(3) **w/o Key**: The key memory window in the Planning Module is removed, and subtask inference relies solely on a single-frame observation.

(4) **GT Classifier**: The Subtask End Classifier is removed, and subtask termination is determined using ground-truth signals provided by the simulator.

Because Mem-0 does not perform subtask decomposition for $M(1)$-type tasks, the ablation study for $M(1)$ includes only the **w/o Anchor** and **w/o Sliding** settings. All ablation results are reported in Table 2.

Analysis on Anchor Memory Performance. Compared to the vanilla setting, removing the anchor memory (**w/o Anchor**) leads to a substantial reduction in success rates across most tasks. Qualitative inspection of evaluation videos indicates that, although the sliding memory window remains active, Mem-0 progressively loses access to task-critical information as relevant memories are evicted over time. As a result, the policy fails to attend to essential cues required for correct decision-making and exhibits erroneous behaviors similar to those observed in Fig. 4. These findings indicate that, for memory-dependent manipulation tasks, it is crucial for a policy to explicitly identify and retain task-critical information throughout execution in order to achieve reliable task completion.

Analysis on Sliding Memory Performance. The Sliding Memory Window primarily captures short-term historical motion trends. Experiments show that, even with the support of anchor information, removing this module still degrades performance across most tasks, with success rates falling below those of the vanilla model. Qualitative results further indicate that, without sliding memory, Mem-0 exhibits unstable and oscillatory behaviors; for example, in the button-pressing task, the policy cannot infer whether the button has already been pressed from a single observation, leading to premature termination or redundant actions and eventual failure.

Interestingly, on the *Swap T* task, the w/o Sliding setting outperforms the vanilla model. This improvement likely stems from the fixed motion patterns in the training data and the task's high sensitivity to the initial orientation of the T-shaped object. Removing sliding memory shifts the policy's reliance toward anchor information and reduces

Figure 4. **Visualization of Baseline Typical Error.** Because the baseline predicts the next action solely from the current observation, it struggles to perform reliably on non-Markovian tasks that require persistent memory over time.

interference from transient motion cues, leading to better performance. This observation suggests that sliding memory can function as either a facilitator or a source of interference, underscoring the importance of coordinating sliding and anchor memories.

Analysis on Key Memory Performance. Under the **w/o Key** setting, where subtask inference is based solely on single-frame observations, success rates decrease substantially relative to the vanilla configuration. Qualitative analysis shows that, for $M(n)$-type tasks requiring long-term information to inform subsequent motion decisions, the Planning Module is unable to reliably infer the correct next subtask when restricted to the current observation alone, resulting in task failure. These results indicate that retaining key memories is critical for accurate and reliable subtask inference in memory-dependent manipulation tasks.

Analysis on the Classifier between the Planning and Execution Modules. The Subtask End Classifier in Mem-0 serves two primary functions. First, it connects the Planning and Execution Modules by triggering high-level reasoning only when necessary, thereby reducing inference cost and latency. Second, it enables task simplification through subtask decomposition. Compared with MemER (Sridhar et al., 2025), which performs high-level subtask reasoning at every timestep, Mem-0 operates at a planning frequency of approximately 5–10 Hz, whereas MemER runs at 1–2 Hz.

In addition, the strong performance achieved with the **GT Classifier** highlights the effectiveness of subtask decomposition relative to fully end-to-end approaches. Nevertheless, the current classifier design in Mem-0 is relatively simple and lacks precision in detecting subtask transitions. Inaccurate transition timing can negatively impact subtask inference in the Planning Module. These results indicate that more refined classifier designs are necessary to achieve tighter coordination between the Planning and Execution Modules through more accurate subtask termination signals.

5.3. Real World Experiment

Table 3. **Real-world Experiment results.**

Tasks	ACT	Pi0.5	Mem-0 (ours)
Put Back Block	0.0%	10.0%	**17.5%**
Rearrange Blocks	0.0%	7.5%	**37.5%**
Cover Blocks	0.0%		
Average	0.00%	8.75%	**27.50%**

To assess the real-world performance of Mem-0, we evaluate it on two physical manipulation tasks aligned with RMBench: Put Back Blocks and Rearrange Blocks. We compare Mem-0 against ACT and Pi0.5, with all data collection and evaluation conducted on the X-One dual-arm robotic platform. For each task, we collect 100 real-world demonstrations and evaluate the trained policies over 40

rollout trials, reporting success rate as the primary metric. The evaluation results are summarized in Table 3.

The results show that Mem-0 outperforms both baseline policies in real-world experiments. Upon closer inspection, we observe that most failures of Mem-0 arise from imprecise block manipulation, rather than high-level task planning. This behavior is likely attributable to two factors. First, real-world data collection involves diverse human behaviors, which introduces additional variability and increases the difficulty of learning consistent low-level manipulation skills. Second, Mem-0 is trained without dedicated pretraining on robotic manipulation, which may limit its ability to generalize fine-grained motor behaviors. Addressing these limitations through improved low-level pretraining and more structured real-world data collection constitutes an important direction for future work.

6. Conclusion

In this paper, we present RMBench (benchmark) and Mem-0 (policy) to systematically evaluate memory in robotic manipulation, revealing the memory limitations of existing policies and how different architectural choices (such as anchor memory, sliding memory, and key memory) affect memory performance, thereby providing preliminary insights into the principled integration of memory mechanisms for effective memory-dependent robotic manipulation.

As for future work, promising directions include improved memory representation and fusion, more robust subtask termination criteria, and the integration of pretraining to enhance semantic understanding and generalization. We hope RMBench fosters principled progress toward scalable, memory-aware robotic manipulation.

Acknowledgements

We would like to thank Xspark AI for supporting our real-world experiments, and D-Robotics for providing the computing resources.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

Bi, H., Tan, H., Xie, S., Wang, Z., Huang, S., Liu, H., Zhao, R., Feng, Y., Xiang, C., Rong, Y., et al. Motus: A unified latent action world model. *arXiv preprint arXiv:2512.13030*, 2025.

Chen, B., Wan, W., Chen, T., Guo, X., Xu, C., Qi, Y., Zhang, H., Wu, L., Xu, T., Li, Z., et al. Univtac: A unified simulation platform for visuo-tactile manipulation data generation, learning, and benchmarking. *arXiv preprint arXiv:2602.10093*, 2026.

Chen, T., Chen, Z., Chen, B., Cai, Z., Liu, Y., Li, Z., Liang, Q., Lin, X., Ge, Y., Gu, Z., et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025a.

Chen, T., Mu, Y., Liang, Z., Chen, Z., Peng, S., Chen, Q., Xu, M., Hu, R., Zhang, H., Li, X., et al. G3flow: Generative 3d semantic flow for pose-aware and generalizable object manipulation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 1735–1744, 2025b.

Cherepanov, E., Kachaev, N., Kovalev, A. K., and Panov, A. I. Memory, benchmark & robots: A benchmark for solving complex tasks with reinforcement learning. *arXiv preprint arXiv:2502.10550*, 2025.

Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11): 1684–1704, 2025.

Fang, H., Grotz, M., Pumacay, W., Wang, Y. R., Fox, D., Krishna, R., and Duan, J. Sam2act: Integrating visual foundation model with a memory architecture for robotic manipulation. *arXiv preprint arXiv:2501.18564*, 2025.

Han, S., Qiu, B., Liao, Y., Huang, S., Gao, C., Yan, S., and Liu, S. Robocerebra: A large-scale benchmark for long-horizon robotic manipulation evaluation. *arXiv preprint arXiv:2506.06677*, 2025.

Intelligence, P., Amin, A., Aniceto, R., Balakrishna, A., Black, K., Conley, K., Connors, G., Darpinian, J., Dhabalia, K., DiCarlo, J., Driess, D., Equi, M., Esmail, A., Fang, Y., Finn, C., Glossop, C., Godden, T., Goryachev, I., Groom, L., Hancock, H., Hausman, K., Hussein, G., Ichter, B., Jakubczak, S., Jen, R., Jones, T., Katz, B., Ke, L., Kuchi, C., Lamb, M., LeBlanc, D., Levine, S., Li-Bell, A., Lu, Y., Mano, V., Mothukuri, M., Nair, S., Pertsch, K., Ren, A. Z., Sharma, C., Shi, L. X., Smith, L., Springenberg, J. T., Stachowicz, K., Stoeckle, W., Swerdlow, A., Tanner, J., Torne, M., Vuong, Q., Walling, A., Wang, H., Williams, B., Yoo, S., Yu, L., Zhilinsky, U., and Zhou, Z. $\pi^*_{0.6}$: a vla that learns from experience, 2025a. URL https://arxiv.org/abs/2511.14759.

Intelligence, P., Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M., Finn, C.,

Fusai, N., Galliker, M. Y., Ghosh, D., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., LeBlanc, D., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Ren, A. Z., Shi, L. X., Smith, L., Springenberg, J. T., Stachowicz, K., Tanner, J., Vuong, Q., Walke, H., Walling, A., Wang, H., Yu, L., and Zhilinsky, U. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025b. URL https://arxiv.org/abs/2504.16054.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

Lan, Z., Jiang, Y., Wang, R., Xie, X., Zhang, R., Zhu, Y., Li, P., Yang, T., Chen, T., Gao, H., et al. Autobio: A simulation and benchmark for robotic automation in digital biology laboratory. *arXiv preprint arXiv:2505.14030*, 2025.

Li, C., Zhang, R., Wong, J., Gokmen, C., Srivastava, S., Martín-Martín, R., Wang, C., Levine, G., Ai, W., Martinez, B., Yin, H., Lingelbach, M., Hwang, M., Hiranaka, A., Garlanka, S., Aydin, A., Lee, S., Sun, J., Anvari, M., Sharma, M., Bansal, D., Hunter, S., Kim, K.-Y., Lou, A., Matthews, C. R., Villa-Renteria, I., Tang, J. H., Tang, C., Xia, F., Li, Y., Savarese, S., Gweon, H., Liu, C. K., Wu, J., and Fei-Fei, L. Behavior-1k: A human-centered, embodied ai benchmark with 1,000 everyday activities and realistic simulation. *arXiv preprint arXiv:2403.09227*, 2024a.

Li, H., Yang, S., Chen, Y., Tian, Y., Yang, X., Chen, X., Wang, H., Wang, T., Zhao, F., Lin, D., et al. Cronusvla: Transferring latent motion across time for multi-frame prediction in manipulation. *arXiv preprint arXiv:2506.19816*, 2025.

Li, Q., Liang, Y., Wang, Z., Luo, L., Chen, X., Liao, M., Wei, F., Deng, Y., Xu, S., Zhang, Y., et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024b.

Li, X., Hsu, K., Gu, J., Pertsch, K., Mees, O., Walke, H. R., Fu, C., Lunawat, I., Sieh, I., Kirmani, S., Levine, S., Wu, J., Finn, C., Su, H., Vuong, Q., and Xiao, T. Evaluating real-world robot manipulation policies in simulation, 2024c. URL https://arxiv.org/abs/2405.05941.

Liang, Z., Li, Y., Yang, T., Wu, C., Mao, S., Nian, T., Pei, L., Zhou, S., Yang, X., Pang, J., et al. Discrete diffusion vla: Bringing discrete diffusion to action decoding in vision-language-action policies. *arXiv preprint arXiv:2508.20072*, 2025.

Lin, M., Ding, P., Wang, S., Zhuang, Z., Liu, Y., Tong, X., Song, W., Lyu, S., Huang, S., and Wang, D. Hif-vla: Hindsight, insight and foresight through motion representation for vision-language-action models. *arXiv preprint arXiv:2512.09928*, 2025.

Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., and Stone, P. Libero: Benchmarking knowledge transfer for lifelong robot learning, 2023. URL https://arxiv.org/abs/2306.03310.

Lu, G., Gao, Z., Chen, T., Dai, W., Wang, Z., Ding, W., and Tang, Y. Manicm: Real-time 3d diffusion policy via consistency model for robotic manipulation. *arXiv preprint arXiv:2406.01586*, 2024.

Mu, Y., Chen, T., Chen, Z., Peng, S., Lan, Z., Gao, Z., Liang, Z., Yu, Q., Zou, Y., Xu, M., et al. Robotwin: Dual-arm robot benchmark with generative digital twins. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 27649–27660, 2025.

Nasiriany, S., Maddukuri, A., Zhang, L., Parikh, A., Lo, A., Joshi, A., Mandlekar, A., and Zhu, Y. Robocasa: Large-scale simulation of everyday tasks for generalist robots. *arXiv preprint arXiv:2406.02523*, 2024.

Shen, W., Liu, Y., Wu, Y., Liang, Z., Gu, S., Wang, D., Nian, T., Xu, L., Qin, Y., Pang, J., et al. Expertise need not monopolize: Action-specialized mixture of experts for vision-language-action learning. *arXiv preprint arXiv:2510.14300*, 2025.

Shi, H., Xie, B., Liu, Y., Sun, L., Liu, F., Wang, T., Zhou, E., Fan, H., Zhang, X., and Huang, G. Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2508.19236*, 2025.

Sridhar, A., Pan, J., Sharma, S., and Finn, C. Memer: Scaling up memory for robot control via experience retrieval. *arXiv preprint arXiv:2510.20328*, 2025.

Su, Y., Zhan, X., Fang, H., Xue, H., Fang, H.-S., Li, Y.-L., Lu, C., and Yang, L. Dense policy: Bidirectional autoregressive learning of actions. *arXiv preprint arXiv:2503.13217*, 2025.

Tao, S., Xiang, F., Shukla, A., Qin, Y., Hinrichsen, X., Yuan, X., Bao, C., Lin, X., Liu, Y., kai Chan, T., Gao, Y., Li, X., Mu, T., Xiao, N., Gurha, A., Rajesh, V. N., Choi, Y. W., Chen, Y.-R., Huang, Z., Calandra, R., Chen, R., Luo, S., and Su, H. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai, 2025. URL https://arxiv.org/abs/2410.00425.

Team, R. Rdt2: Enabling zero-shot cross-embodiment generalization by scaling up umi data, September 2025. URL https://github.com/thu-ml/RDT2.

Wang, Y., Wu, R., Chen, Y., Wang, J., Liang, J., Zhu, Z., Geng, H., Malik, J., Abbeel, P., and Dong, H. Dexgarmentlab: Dexterous garment manipulation environment with generalizable policy, 2025. URL https://arxiv.org/abs/2505.11032.

Wen, J., Zhu, Y., Zhu, M., Tang, Z., Li, J., Zhou, Z., Liu, X., Shen, C., Peng, Y., and Feng, F. Diffusionvla: Scaling robot foundation models via unified diffusion and autoregression. In *Forty-second International Conference on Machine Learning*.

Wen, J., Zhu, M., Zhu, Y., Tang, Z., Li, J., Zhou, Z., Li, C., Liu, X., Peng, Y., Shen, C., and Feng, F. Diffusionvla: Scaling robot foundation models via unified diffusion and autoregression. *arXiv preprint arXiv:None*, 2024.

Wen, J., Zhu, Y., Li, J., Tang, Z., Shen, C., and Feng, F. Dexvla: Vision-language model with plug-in diffusion expert for general robot control. *arXiv preprint arXiv:2502.05855*, 2025a.

Wen, J., Zhu, Y., Li, J., Zhu, M., Tang, Z., Wu, K., Xu, Z., Liu, N., Cheng, R., Shen, C., et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025b.

Xiang, F., Qin, Y., Mo, K., Xia, Y., Zhu, H., Liu, F., Liu, M., Jiang, H., Yuan, Y., Wang, H., et al. Sapien: A simulated part-based interactive environment. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 11097–11107, 2020.

Ze, Y., Zhang, G., Zhang, K., Hu, C., Wang, M., and Xu, H. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. *arXiv preprint arXiv:2403.03954*, 2024.

Zhao, T. Z., Kumar, V., Levine, S., and Finn, C. Learning fine-grained bimanual manipulation with low-cost hardware, 2023. URL https://arxiv.org/abs/2304.13705.

Zheng, J., Li, J., Wang, Z., Liu, D., Kang, X., Feng, Y., Zheng, Y., Zou, J., Chen, Y., Zeng, J., Zhang, Y.-Q., Pang, J., Liu, J., Wang, T., and Zhan, X. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model, 2025. URL https://arxiv.org/abs/2510.10274.

Zheng, Y., Zhang, R., Zhang, J., Ye, Y., Luo, Z., Feng, Z., and Ma, Y. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand, 2024. Association for Computational Linguistics. URL http://arxiv.org/abs/2403.13372.

A. RMBench Tasks Description

Table 4. **Task descriptions of RMBench benchmark.**

Task	**Description**
Observe and Pick Up	A reference object is placed on a shelf, and multiple objects are placed on the table. The robot first observes the reference object while remaining stationary. After the reference object is hidden, the robot must pick up the matching object from the table.
Rearrange Blocks	Two pads and a button are placed on the table. One block is positioned between the two pads, and another block is placed on one of the pads. The robot moves the middle block onto a pad, presses the button, and then moves the other block to the middle position.
Put Back Block	Four pads are arranged around a central position, with one block placed on one of the pads. The robot moves the block to the center, presses the button, and then returns the block to its original pad.
Swap Block	Three pads and a button are placed on the table, with two blocks placed on different pads. The robot uses the empty pad to swap the positions of the two blocks and then presses the button.
Swap T	Two T-shaped blocks with different colors are placed on the table. The robot picks up both blocks and swaps their positions and orientations.
Battery Try	Two batteries with random orientations and a dual-slot battery holder are placed on the table. The robot repeatedly attempts different insertion orders, placing both batteries into the holder with the correct orientations until the insertion succeeds.
Blocks Ranking Try	Three blocks of different colors are randomly arranged on the table, along with a button. The robot repeatedly attempts different block arrangements and presses the button to confirm until the correct ordering is achieved.
Cover Blocks	Three colored blocks (red, green, and blue) and three covers are placed on the table. The robot covers the blocks from left to right, then uncovers them in red–green–blue order and returns the covers to their original positions.
Press Button	Three buttons (left, middle, and right) and two single-digit number tiles are placed on the table. The robot presses the left button the number of times indicated by the left digit, presses the middle button the number of times indicated by the right digit, and then presses the right button to confirm.

B. Training Details

B.1. Planning Module

In the Planning Module, we fine-tune the vision–language model (Qwen3-VL-8B-Instruct) using LoRA via LLaMA-Factory (Zheng et al., 2024) to enable reasoning over key memories. After fine-tuning, we deploy the model with vLLM (Kwon et al., 2023) for efficient loading and inference. The key hyperparameters used for VLM fine-tuning are summarized in Table 5. Training is conducted on 8 NVIDIA A800 GPUs and the duration of training for a single task is approximately half an hour.

Table 5. **Hyperparameters for fine-tuning the Mem-0 Planning Module.**

Configuration	Finetuning Type	LoRA Rank	Batch Size	Learning Rate	Epochs	LR Scheduler	Warmup Ratio	Dtype
Value	LoRA	8	16	1.0×10^{-4}	25	Cosine	0.1	bf16

B.2. Execution Module

In this section, we detail the training infrastructure, training organization strategy and hyperparameter configurations employed for Execution Module in Mem-0.

B.2.1. Training Infrastructure and Time Budget

The Execution Module of Mem-0 utilizes a single-task training strategy, where the model is trained from scratch for each specific task. Training is conducted on 8 NVIDIA A800 GPUs with a global batch size of 448 over 30K iterations. The duration of training for a single task is approximately 18 hours.

B.2.2. Training Organization Strategy

Forward Pass Strategy. Given the memory-centric architecture of Mem-0, we employ a specialized training methodology. Within each batch, the processes of VLM token generation and DiT-based action chunk generation are executed in parallel. Conversely, the fusion of Sliding Memory and Anchor Memory requires the temporal integrity of the data; therefore, this stage is processed serially, ensuring that all frames within an episode remain sequentially aligned.

This approach guaranties the effective utilization of VLM tokens. Furthermore, we maintain a global data structure during training to store memory information for each episode, facilitating seamless cross-batch token utilization.

Dataloader Implementation. Consequently, the dataloader was custom-designed to align with this architecture. Episodes are distributed as evenly as possible across all GPUs. Each GPU processes its assigned episodes using a specific number of workers and manages the dataloader reset independently. Due to the stochastic nature of the distribution, as training iterations progress, the frames within a global batch become temporally desynchronized. This allows the model to learn simultaneously from data that span various time steps.

B.2.3. Hyperparameter Configurations

Table 6. summarizes the key training hyperparameters. To balance the learning rate requirements of different modules, we implemented a grouped learning rate strategy alongside a cosine learning rate schedule with linear warm-up. Regarding numerical precision, the VLM and Memory Bank components operate in `bfloat16`, while all other modules utilize `float32`. Furthermore, regarding visual inputs, images are resized to 224×224 and subjected to mild data augmentation via frame-independent `ColorJitter`, aimed at enhancing the model's generalization capabilities.

Table 6. **Hyperparameters for training the Mem-0 Execution Module.**

Configuration	Value
Batch Size	448
Iterations	30,000
Max Grad. Norm	2.5
LR Scheduler	Cosine
Warmup Ratio	0.05
Optimizer	AdamW
Momentum	$\beta_1, \beta_2 = 0.9, 0.999$
Weight Decay	0.005
Image Resize	224×224
Image Aug.	ColorJitter[†]

[†] Jitter: (0.1, 0.1, 0.1, 0)

Configuration	Value
LR (Base)	1.0×10^{-5}
LR (VLM)	1.0×10^{-5}
LR (Action Head)	1.0×10^{-4}
LR (Classifier)	1.0×10^{-4}
Min LR (Base)	1.0×10^{-6}
Min LR (VLM)	1.0×10^{-6}
Min LR (Action Head)	5.0×10^{-6}
Min LR (Classifier)	5.0×10^{-6}
Workers per GPU	2

C. Additional Visualizations and Analysis of Failure Cases in Mem-0

While Mem-0 demonstrates substantial improvements over the baselines, its architectural design still offers extensive room for further exploration. In this section, we present representative cases where Mem-0 exhibits suboptimal performance, aiming to provide valuable insights for future research.

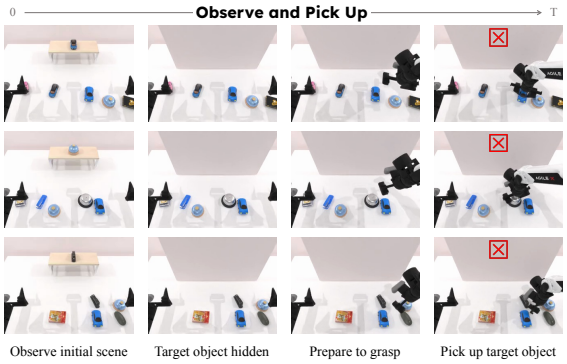

Figure 5. **Failure examples of Observe and Pick Up**. **(Top)** Confused by objects with similar colors and shapes. **(Middle)** Confused by identical object morphologies. **(Bottom)** General failure to identify the target, resulting in the robot grasping a mean position or unintended position.

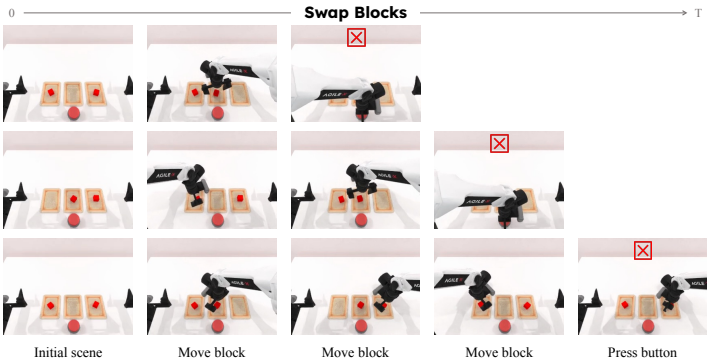

Figure 6. **Failure examples of Swap Blocks**. **(Top)** Premature termination after a single subtask. **(Middle)** Premature termination after two subtasks. **(Bottom)** Failure to terminate on time, resulting in the initiation of a redundant subtask.

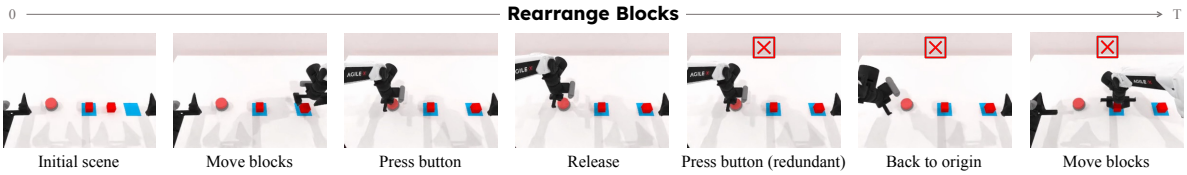

Figure 7. **Failure examples of Rearrange Blocks**. Mem-0 redundantly presses the button, resulting in task failure.

C.1. Failures Analysis for $M(1)$ Tasks

For $M(1)$ tasks, in addition to the failures illustrated in Fig. 4, we summarize the representative errors encountered by Mem-0 below.

Observe and Pickup & Swap Blocks. Fig. 5 illustrates typical failure modes in the *Observe and Pick Up* task, where Mem-0 fails to accurately identify the target object. Similarly, Fig. 6 presents examples of misjudgments regarding the termination of the swapping sequence in the *Swap blocks* task, resulting in the confirmation button being pressed at inappropriate timings.

These instances reveal that the Anchor Memory, in fact, exerts a continuous influence throughout the entire task horizon. Consequently, the model must maintain constant attention to the Anchor Memory and intelligently modulate the degree of its contribution to action prediction.

Nevertheless, our ablation studies have already demonstrated the substantial performance gains brought by the Anchor Memory within the Mem-0 architecture, with its impact being particularly pronounced in tasks such as *Rearrange Blocks* and *Put Back Block*.

Rearrange Blocks. On the other hand, Fig. 7 illustrates failure cases in the *Rearrange Blocks* task where Mem-0 performs excessive button presses, a behavior we attribute to the limitations of the Sliding Memory module. Quantitative results from our ablation studies show a significant performance degradation in this task when the Sliding Memory is omitted. By analyzing the video playbacks, we found that the frequency of redundant button-pressing events increases markedly without Sliding Memory, identifying it as a primary failure mode. These findings demonstrate that while the current Sliding Memory provides substantial performance gains, there remains potential for further refinement.

Summarize. To address these observations, we believe that one potential avenue for enhancement involves exploring more richer representation fusion mechanisms to improve the utilization of both Anchor and Sliding Memory. Such advances would further improve the performance and stability of the model in more intricate and versatile scenarios. Additionally, increasing the visual processing capabilities of existing VLM architectures is expected to yield better results in tasks such as *Observe and Pick Up*.

C.2. Failures Analysis for $M(n)$ Tasks

For $M(n)$ tasks, although Mem-0 demonstrates substantial improvements over the baselines, there remains significant room for further improvement. We believe that the primary challenge to be addressed lies in the performance and robustness of the Classifier.

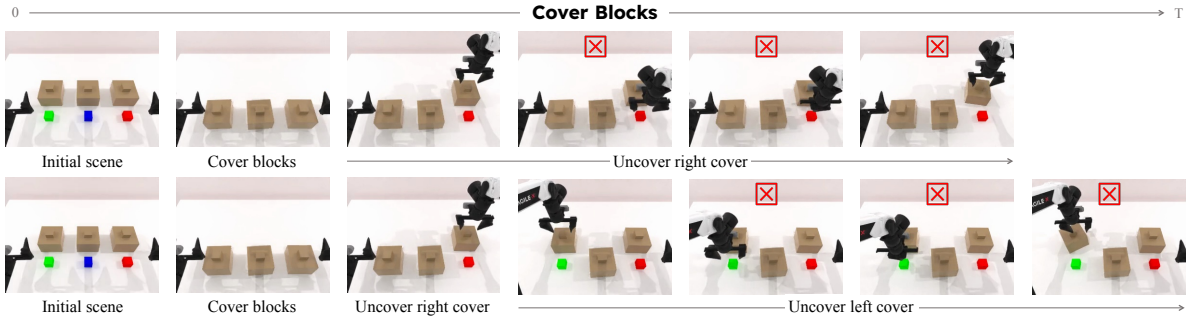

Figure 8. **Failure examples of Cover Blocks**. The Classifier fails to accurately detect the completion of the *Uncover xxx* subtask, thereby preventing a subtask transition. As the instruction remains unchanged, the model is forced to operate under a wrong task context, leading to unintended and erratic behaviors.

Cover Blocks. The current design of Classifier occasionally fails to accurately perceive the ongoing task progress, leading to an inability to discriminate whether a specific state represents the initiation or the termination of a subtask. This phenomenon is exemplified in Fig. 8 during the execution of the *Cover Blocks* task.

Blocks Ranking Try. Another primary challenge pertains to the execution of button-pressing operations. We observe that the inclusion of button-pressing actions often introduces interference into hybrid tasks that are not exclusively focused on pressing. For instance, in *Blocks Ranking Try*, the transition between button-pressing and block-swapping is occasionally fluidly disrupted, a phenomenon illustrated in Fig. 9.

In the *Blocks Ranking Try* task, even a single execution error inevitably leads to overall task failure. This sensitivity is the main driver of failure for this task, as clearly substantiated by the quantitative results of our ablation studies.

Figure 9. **Failure examples of Blocks Ranking Try**. Upon pressing the button, the system is expected to transition to the next subtask to execute the swapping of designated blocks. However, the Classifier fails to trigger this transition promptly, causing the task to stall in the `Press button` state. This leads to a coordination conflict between the dual arms: the right hand attempts to initiate manipulation while the left hand remains tethered to the button-pressing instruction.

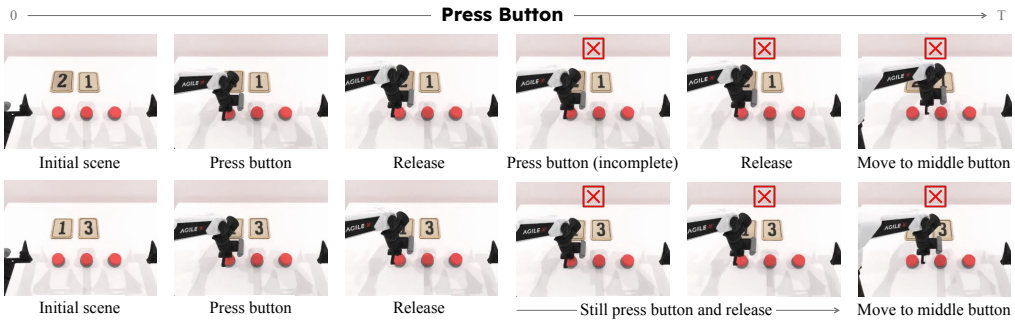

Figure 10. **Failure examples of Press Button**. **(Top)** Insufficient presses: the Classifier issues a false positive termination signal even when the button-press is unsuccessful. **(Bottom)** Excessive presses: the Classifier fails to recognize a successful subtask completion, leading to redundant execution of the same subtask.

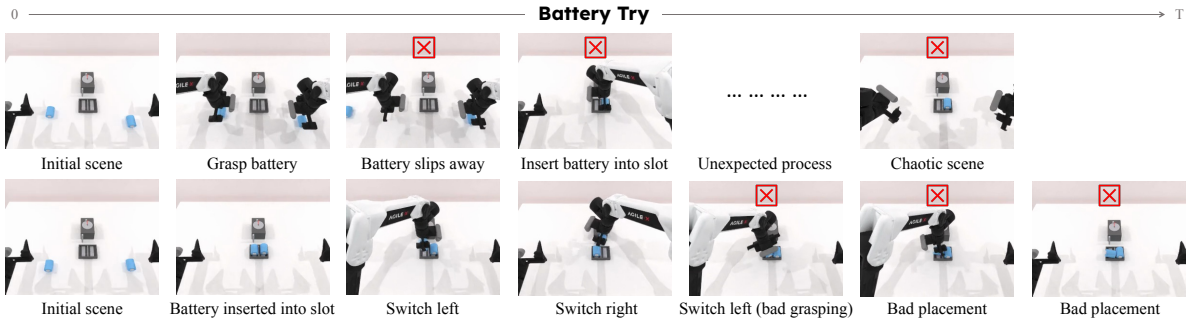

Figure 11. **Failure examples of Battery Try**. **(Top)** For the horizontally oriented cylindrical battery, a suboptimal grasp pose prevents a successful lift and causes significant displacement, leading the model into unforeseen observational states. **(Bottom)** The model fails to commit to a specific manipulation strategy during battery adjustment, resulting in a mean action that leads to improper placement in the slot.

Press Button. Specifically, the *Press Button* task, as a dedicated button-pressing scenario, underscores the inherent difficulty of this operation. In the current design of Mem-0, the state of the button (pressed vs. unpressed) is reflected in the visual input only through extremely subtle cues. Consequently, the visual tokens generated by the VLM backbone lack the granularity to encapsulate such fine-grained information, creating a fundamental bottleneck for the downstream Classifier. Fig. 10 displays representative failure cases of the Classifier in the *Press Button* task.

Battery Try. Furthermore, for the *Battery Try* task, a representative failure mode is suboptimal manipulation precision. This encompasses both insertion errors when placing the battery into the slot and challenges in determining the appropriate grasp strategy due to the extremely subtle visual cues of the slot, as illustrated in Fig. 11.

Summarize. To address these observations, we posit that incorporating proprioceptive or tactile feedback could provide the Classifier with critical non-visual information. Moreover, as the Classifier is the most downstream module in Mem-0, optimizing upstream VLM token extraction and the fusion mechanisms of Anchor and Sliding Memory remains a promising avenue. Such refinements would allow the Classifier to operate on input tokens with better-conditioned distributions and enhanced informational saliency. We also anticipate that more interpretable tokens could improve the synergy between diverse subtasks, thereby further enhancing performance in complex scenarios like *Blocks Ranking Try*.

Nevertheless, the efficacy of the current design has been validated across various $M(n)$ tasks, notably yielding substantial performance gains in *Cover Blocks* compared to the baseline. We hope that the aforementioned analysis provides valuable insights for future work to further improve performance.